

Angular CLI : Project Setup

By CODEMIND Technology

Contact us +91 96650 44698

Angular CLI

The Angular CLI is a command line interface tool that can create a project, add files, and perform a variety of ongoing development tasks such as testing, bundling, and deployment.

Development Environment

Editor : Visual Studio Code

<https://code.visualstudio.com/download>

Node Js : v16.14.0 (node -v)

NPM : 8.3.1 (npm -v)

Install Angular CLI

`npm install -g @angular/cli@14.0.6 globally`

`:> ng new angularapp`

`:>ng version`

Steps:

ng new my-app

cd my-app

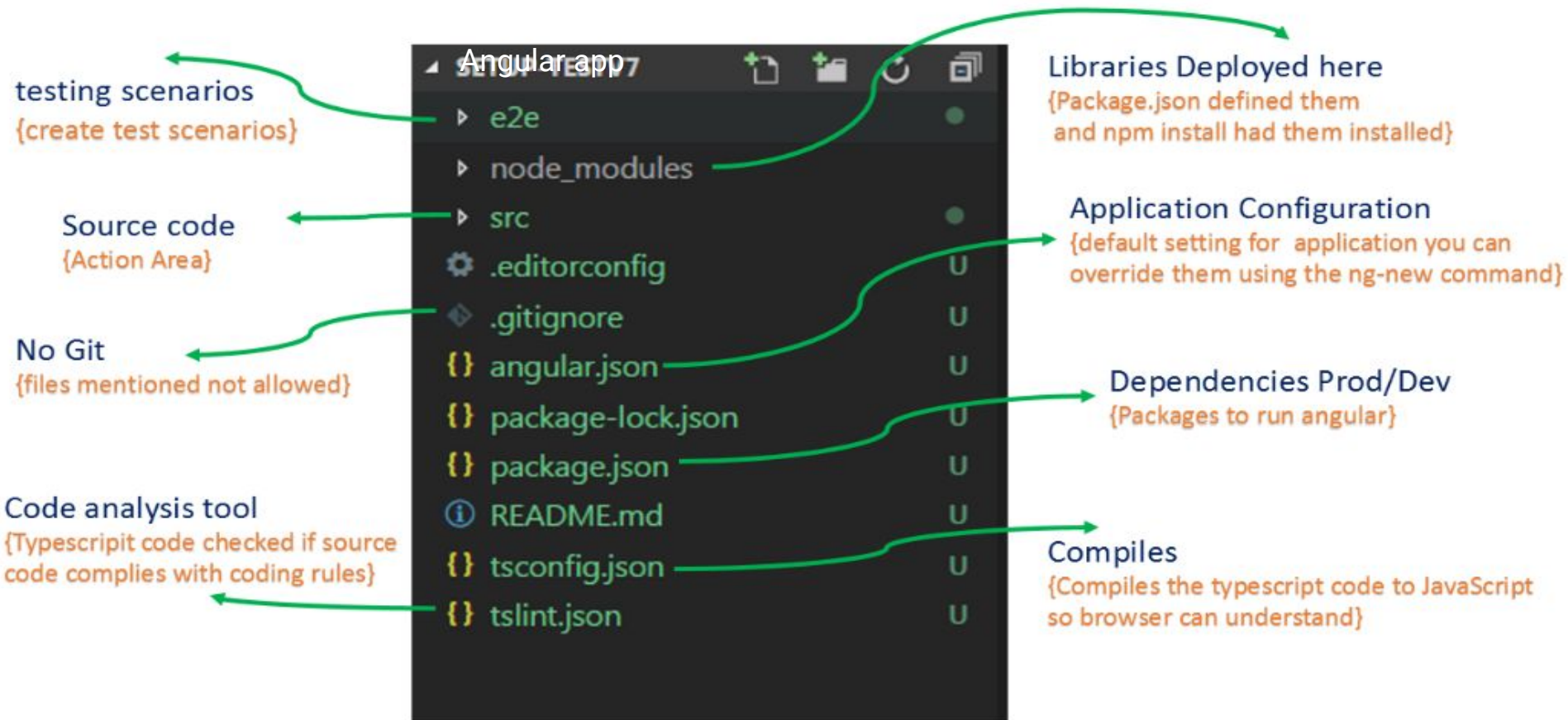
ng serve

generate a new project

go to the project directory

launches the server

Angular folder structure



Folder Structure of Angular Apps :

1. Project name i.e angularapp

2. e2e

- protractor framework this comes by default.

e2e is nothing but end to end test scripts most of the automation scripts written by the qa engineer.

`protractor.config.js`

-> configuration settings for running the end to end test scripts.

src Folder

We keep our source code like component, modules, templates, services, directives, images etc.

Entire source code of your application is inside src folder.

Things we will look here:-

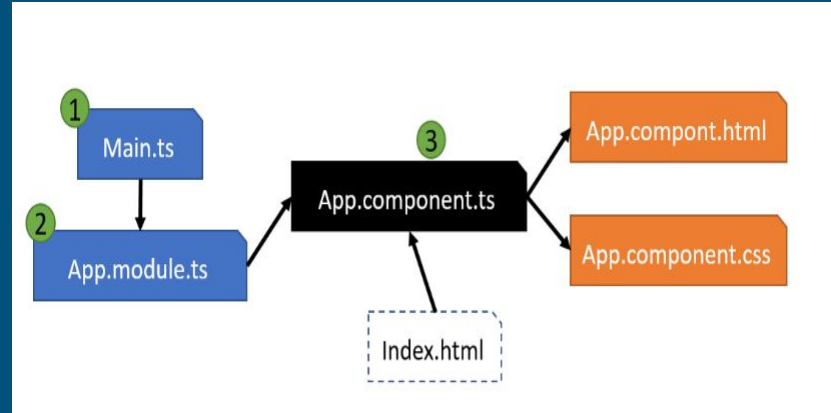
- app Folder
- environments Folder
- Index.html
- Main.ts & Polyfills.ts

app Folder

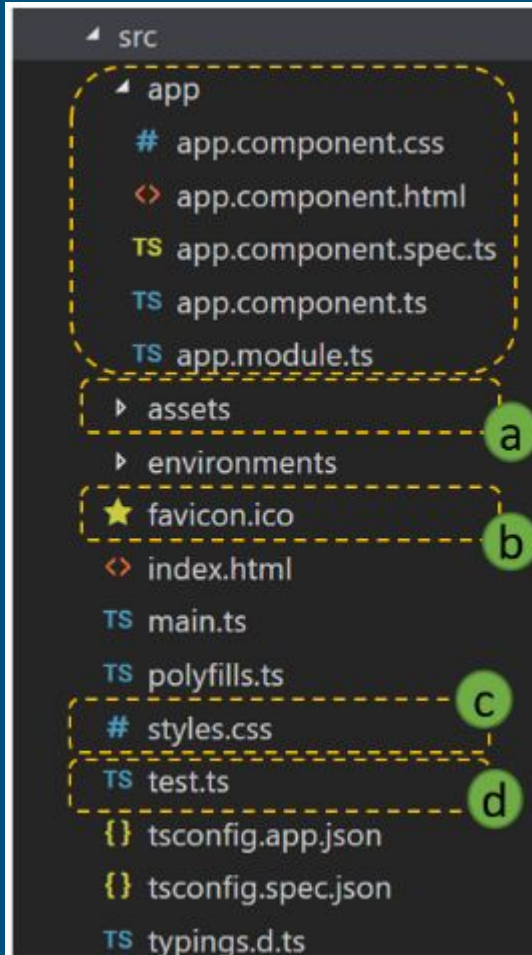
The sub-folder /app/ under /src/ contain our modules and component. Every angular application comes up with at-least one module and component.

- Module groups component which are dedicated for the functioning of an application. The module file is named as app.module.ts and there is at-least one module inside an angular application.
- While the component defines the view (things you see on browser) and the application logic. A module can have lots of component.

Flow inside angular



- 1) Main.ts the entry point from which
- 2) app.Module.ts gets called which in turns call a particular
- 3) App.component.ts file.



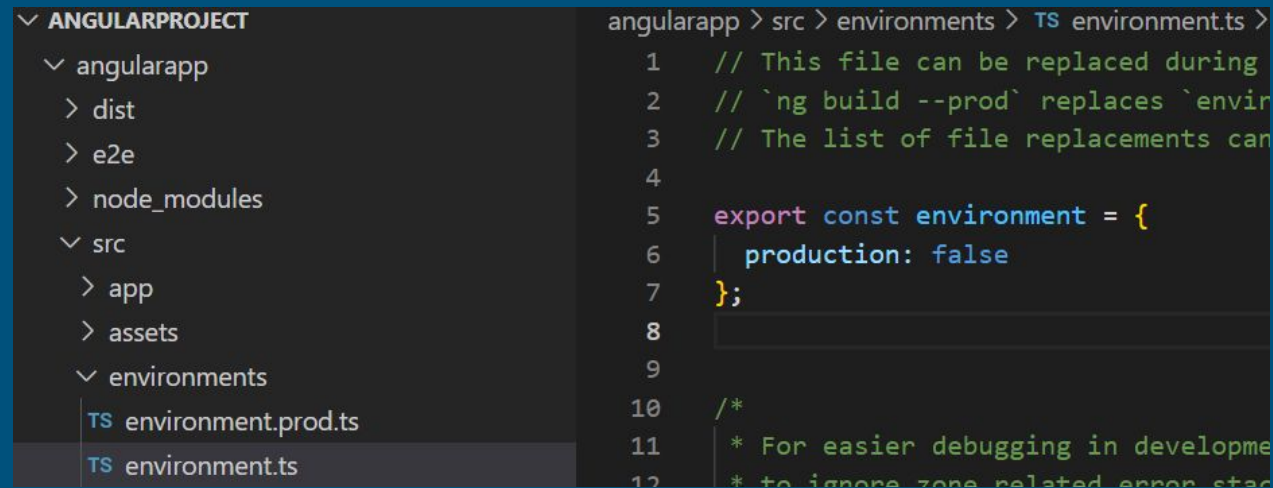
- a) assets : Store's all the static files like Images and icons etc. used within your application
- b) favicon.ico : The icon for your App
- c) style.css : Global CSS and stylesheet which can be used by all your pages and components
- d) test.ts : Configuration used for setting the testing environment

environments Folder

The sub-folder /environments/ under /src/ contain 2 files each for different environment and they are used to store configuration information like database credential, server address, or some property that differs across those environments.

1) environment.ts: This environment file is used by default and is used during the development. If you open this file you will notice that production property is set to false.

2) environment.prod.ts: It is used during the production build, this is called when you build your application with `ng build --env=prod`.



```
angularapp > src > environments > TS environment.ts >
1  // This file can be replaced during
2  // `ng build --prod` replaces `envir
3  // The list of file replacements can
4
5  export const environment = {
6    production: false
7  };
8
9
10 /*
11  * For easier debugging in developme
12  * to ignore zone related error sta
```

Index.html

This is the file called by the user. A plain html file which contains your angular directive (e.g. `<app-root></app-root>`) which calls the angular component.

The diagram illustrates the structure of an Angular application. It consists of three main parts:

- File Explorer (Left):** Shows the project structure. The file `index.html` is highlighted with a dashed yellow box and labeled with a green circle '1'.
- index.html (Top Right):** Shows the HTML structure. The `<app-root></app-root>` directive is highlighted with a dashed yellow box and labeled with a green circle '2'.
- app.component.ts (Bottom Right):** Shows the TypeScript code for the `AppComponent`. The `@Component` decorator is highlighted with a dashed yellow box, and an arrow points from this box to the `<app-root></app-root>` directive in the `index.html` file.

```
src
├── app
│   ├── app.component.css
│   ├── app.component.html
│   ├── app.component.spec.ts
│   ├── app.component.ts
│   └── app.module.ts
├── assets
├── environments
├── favicon.ico
├── index.html
├── main.ts
├── polyfills.ts
├── styles.css
├── test.ts
└── tsconfig.json
```

```
<meta name="viewport" content="width=device-width" />
<link rel="icon" type="image/x-icon" href="favicon.ico" />
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

```
TS app.component.ts x
1  import { Component } from '@angular/core';
2
3  @Component({
4    selector: 'app-root',
5    templateUrl: './app.component.html',
6    styleUrls: ['./app.component.css']
7  })
8  export class AppComponent {
```

Main.ts & Polyfills.ts

Main.ts :- This is the starting point of the app. It is here your application get compiled with Just In Time compiler and then it bootstraps or load the Module called AppModule which is the root module of the application.

Every angular application has at-least one Angular Module called the root module which is called the AppModule.

```
TS main.ts  x
4  import { AppModule } from './app/app.module';
5  import { environment } from './environments/environment';
6  |
7  if (environment.production) {
8  |   enableProdMode();
9  | }                                     Bootstrapping AppModule
10 |
11 | platformBrowserDynamic().bootstrapModule(AppModule)
12 |   .catch(err => console.log(err));
13 |
```

Polyfills.ts

Angular is built on the latest standards of the web platform which target a wide range of browsers and this becomes challenging to ensure things do not break in the old browsers.

Polyfills are workaround scripts that ensure that your recent codes (which use the new browser features) do not break in the old browsers (which do not support the new browser features).

Angular.json

Angular.json at the root level of an Angular workspace provides workspace-wide and project-specific configuration defaults for build and development tools provided by the Angular CLI.

```
angular.json X simpletemplate.component.html reactive.component.html
angularapp > {} angular.json > {} projects
1  {
2    "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
3    "version": 1,
4    "newProjectRoot": "projects",
5  > "projects": { ...
122 |   },
123   "defaultProject": "angularapp"
124 }
```

Angular json

- \$schema: used by Angular CLI in order to enforce the specification of Angular workspace schema , this \$schema property refers to angular CLI implementation file.
- Version: The configuration-file version.
- newProjectRoot: Path where new projects are created.
- projects: Contain a subsection for each project (library or application) in the workspace.
- defaultProject: When you use `ng new` to create a new app in a new workspace, this app gets created as a default project in the workspace.

Package.json

This contains list of packages required to build and run our angular application. There are two lists of such packages called dependencies and dev dependencies.

dependencies: Normal project dependency which are both required during runtime and development. You can think of them as a reference to mandatory modules.

devDependencies : These modules are only required during the development and they don't get called actual project release or runtime.

This file also contains some basic information about project like (name, description, license etc). The interesting part here is the scripts section which takes Key/Value as input. Each one of the keys in these key/value pairs is the name of a command that can be run and the corresponding value of each key is the actual command that is run.

tsconfig.json

Typescript is the primary language for Angular application but the problem is that browsers don't understand this language, so the typescript must be transpiled into the Javascript.

Hence this file defines the compiler option required to compile the project.

Thank you

Contact us - 96650 44698

